React.js Internal Coding Standard - IT Path Solutions

---

1. Introduction

- **Purpose**

  This document defines **mandatory coding standards and architectural rules** for all React-based projects within the organization. Its goal is to ensure **consistency, scalability, maintainability, and performance** across teams and projects.

- **Scope**

  These standards apply to:

  - Web applications built using React
  - Projects using TypeScript or JavaScript
  - Internal tools, dashboards, and customer-facing apps

- **Supported projects**

  - React + Vite
  - React + CRA (legacy only)
  - React + Webpack (custom builds)

- **Audience**

  - Frontend Developers
  - Full-stack Developers
  - Tech Leads & Architects
  - Code Reviewers

---

2. Prerequisites & Setup

- **Node.js version requirement**

    - Minimum Required: Node.js ≥ 20.19 or higher, or 22.12 or higher.
    - Preferred Version: Node.js 22.x or current LTS
    - Environment Consistency:

    - The Node.js version must be identical across:
        - Local development
        - CI pipelines
        - Production environments

- **Must be aligned across local, CI, and production environments**

  Step 1: The `.nvmrc` file must exist at the project root and define the Node.js version to be used across all environments. The file **must specify either a major version** (e.g., `22`) **or an exact version** (e.g., `22.11.0`), depending on the required level of version strictness.

  Step 2: `package.json` Engine Enforcement

    - The `package.json` file **must** declare the supported Node.js versions using the `engines` field.

```JSON
{
  "engines": {
    "node": ">=18 <23"
  }
}
```

  Step 3: `Local Development`

    - Developers **must** use `nvm` (or an equivalent Node version manager).

```bash
nvm install
nvm use
```

Step 4: CI Configuration

- CI pipelines **must** read the Node.js version directly from `.nvmrc`
  Example (GitHub Actions):

```yaml
- uses: actions/setup-node@v4
  with:
    node-version-file: '.nvmrc'
```

Step 5: Production Environment

- Production servers **must** use `nvm` (or Volta) and load the Node.js version from `.nvmrc`

```bash
nvm install
nvm use
```

- **Package manager**
  - Preferred: `npm or bun`
  - Allowed: `pnpm` or `yarn` (project-dependent)
  - Lock files must be committed
- **Environment setup**

- `.env.example` is mandatory
- No secrets allowed in `.env` files committed to git
- Environment variables must be accessed via a centralized config

- **Build tool (Vite / CRA / Webpack)**
    - Default: Vite
    - CRA is allowed only for legacy maintenance
    - Custom Webpack requires architecture approval

---

3. Boilerplate Reference

- **Internal starter repository**
    - All new projects must start from the internal React starter
    - Custom setups require approval
    - Starter Repo Link

- **Branch rules**

```
Branch → Environment Mapping

main        → production
develop     → staging
feature/*   → development
fix/*       → development
hotfix/*    → production
```

- **Versioning**
    - Semantic Versioning (`MAJOR.MINOR.PATCH`)
    - Tags are mandatory for production releases

---

4. Project Structure

Example:

/src

/assets

- **Purpose:** Static assets such as images, icons, and fonts.
- **Example:**

```
/assets
├─ images
│  ├─ logo.svg
│  └─ banner.png
│
└─ icons
   ├─ index.ts
   └─ icon.tsx
```

/components/ui (Pure UI components for shadcn)

- **Purpose:** Reusable, framework-level UI primitives (e.g., shadcn-style components).
- **Example:**

```
/components/ui
  |  input.tsx
  └─ button.tsx
```

/components

- **Purpose:** Feature-level pure UI components (no logic, no API calls).
- **Example:**

```
/components/user-management
  |  user-action.tsx
  └─ user-card.tsx
```

/containers

- **Purpose:** Smart components handle data, provide hooks, and coordinate screens.

- **Example:**

```
containers/user-management/
├─ users/
│  └─ index.tsx
├─ add-edit-user/
│  └─ index.tsx
```

`/integrations` (3rd party integrations like Stripe, GA4, google-map-service, etc)

- **Purpose:** Third-party SDK and platform integrations.
- **Example:**

```
/integrations/ga4
  └─ index.ts
```

`/shared` (Project-specific custom components)

- **Purpose:** Project-specific reusable components that are not generic UI primitives.
- **Example:**

```
/shared/empty-state
  └─ empty-state.tsx
```

`/hooks` (Global hooks)

- **Purpose:** Global, reusable hooks not tied to a specific feature.
- **Example:**

```
/hooks
 ├─ use-local-storage.ts
 └─ use-debounce.ts
```

`/layouts` (App layouts)

- **Purpose:** Application-level layout components.
- **Example:**

```
/layouts
  ├─ auth-layout.tsx
  └─ dashboard-layout.tsx
```

`/pages` (Route-level pages)

- **Purpose:** Route-level entry points. Pages must only mount containers.
- **Example:**

```
/pages/user-management
  └ index.tsx
```

/routes (Route configuration)

- **Purpose:** Centralized route configuration.
- **Example:**

```
/routes
  ├ routes.ts
  ├ index.tsx
  ├ public.routes.tsx
  └ private.routes.tsx
```

/services (API configuration, Interceptors, and Business services)

- **Purpose:** API configuration, interceptors, and business services.
- **Example:**

```
/services
  ├ index.ts
  └ api-client.ts
```

/store

- **Purpose:** Global state management (Redux / Zustand).
- **Example:**

```
/store
  ├ slice
  |   ├ auth.slice.ts
  |   ├ users.slice.ts
  |   ├ products.slice.ts
  |   ├ index.ts
  └ index.ts
```

/styles (Global styles)

- **Purpose:** Global styles and theme configuration.
- **Example:**

```
/styles
  └ globals.css
```

/types

- **Purpose:** Shared global TypeScript types and interfaces.
- **Example:**

```
/types
  ├─ auth.types.ts
  └─ user-access.types.ts
```

`/utils`

- **Purpose:** Helper functions, constants, and utility logic.
- **Example:**

```
/utils
├─ functions
├─ validations
└─ constants
```

Rules:

- No API calls inside `/components`
- No business logic inside `/pages`
- Each folder has a single responsibility

Note: `/styles` - This directory must not be created when using shadcn/ui with Tailwind CSS.
It may be added only for project-specific global styles, theme overrides, or exceptional styling requirements.

---

5. Coding Conventions

- **Naming Rules**
    - Components: `PascalCase`
    - Hooks: `useCamelCase`
    - Variables & functions: `camelCase`
    - Constants: `UPPER_SNAKE_CASE`
    - Files match exported entity names
    - Use `kebab-case` for file names
- **Formatting (ESLint + Prettier)**
    - ESLint + Prettier are mandatory
    - Preferred: `husky or pre-commit hooks`
    - No manual formatting overrides

    - Lint must pass in CI

- **TypeScript Rules**

    - `strict: true` is mandatory
    - `any` is not allowed
    - Use `unknown` instead of `any`
    - `Explicit return types for public functions`
    - `interfaces for objects, types for unions`

---

6. Layout Management

Layout Types

Layouts define **structural UI**, not business logic.

Supported layouts:

- Base layout - Public pages (landing, marketing)
- Auth layout - Login, register, reset password
- Dashboard layout - Authenticated application shell
- Nested layouts
    - Nested layouts are allowed for dashboards and sub-modules
    - Avoid deep nesting (>2 levels)
- Responsive rules
    - Mobile-first approach
    - No hard-coded widths
    - Use design breakpoints consistently

- Layouts must be tested on: (Desktop, Tablet, and Mobile)

---

7. Route Management & Best Practices (React Router)

7.1 Route Structure

```
/routes
  index.tsx
  auth.routes.tsx
  private.routes.tsx
```

Rules:

- Routes are declared centrally
- Pages **must not** self-register routes

7.2 Public Routes

Examples:

```
/home
/terms-and-conditions
/privacy-policy
```

Rules:

- No authentication required
- Authenticated (logged-in) users must be redirected to the appropriate protected route (e.g., dashboard)
- Data fetching must be handled via services, hooks, or server-side logic (if applicable)

7.3 Private Routes

Examples:

```
/dashboard
/profile
/settings
/orders, etc.
```

Use wrapper:

```typescript
<ProtectedRoute>
  <Dashboard />
</ProtectedRoute>
```

Rules:

- Authentication logic is centralized
- No token checks inside pages
- Redirect logic is handled only by `ProtectedRoute`

7.4 Auth Routes

Examples:

```
/auth/login
/auth/register
/auth/forgot-password, etc.
```

Use wrapper:

```typescript
<AuthRoute>
  <Login />
</AuthRoute>
```

Rules:

- Routes are accessible only to unauthenticated users
- All authentication logic and API interactions must be handled via services or hooks
- Redirect logic is dealt with only by the `AuthRoute wrapper`

7.5 Query-Supported Routes

Examples:

```
/users?page=1&limit=10
/purchase?payment=success
```

Rules:

- Use `useSearchParams (You can make your own hook using react-router)`
- Validate query params
- Sync UI state with URL
- Never rely on unvalidated query values

7.6 Dynamic Routes

Examples:

```
/users/:id
/products/:slug
```

Rules:

- Validate route params
- Handle loading & error states
- Never assume the params' existence

---

8. State Management

Use Case	Solution
Component-only state	`useState`
Shared UI state	Context
App-wide state	Redux / Zustand
Server state	React Query / TanStack Query

- **Store structure**
    - **Zustand (Recommended)**

```
Zustand – Global Store Structure

store/
├── index.ts              → Exports all global Zustand stores
├── auth.store.ts         → Manages global authentication state
├── theme.store.ts        → Manages global theme state (light/dark/system)
```

index.ts (Aggregator Pattern)

```typescript
export * from "./auth.store";
export * from "./theme.store";
```

Rules:

- The `store` folder must contain **only global-level Zustand stores**
- Global stores represent cross-application concerns (e.g., auth, theme, app config)
- Feature-specific or page-level state must **not** be added here
- Each store must be **independent and modular**

- UI components must consume state via the exported Zustand hooks only

- **Redux Toolkit (RTK)**

```
RTK (Redux Toolkit) – Global Store Structure

store/
├── index.ts                → Main Redux store configuration
├── slices/
│   ├── index.ts            → Combines all global slices and creates the root reducer
│   ├── auth.slice.ts       → Manages global authentication state
│   ├── theme.slice.ts      → Manages global theme state (light/dark/system, etc.)
```

Rules:

- The `slices` folder must contain only global-level slices
- Global slices represent **cross-application concerns** (e.g., auth, theme, app config)
- Feature-specific or page-level state must **not** be added here
- All slices must be imported and combined **only** in `slices/index.ts`

---

9. API & Networking

API Architecture Overview

The application must follow a **two-layer API architecture** inside the `/services` directory:

```
Plain text

/services
  client.ts   // Axios client, interceptors, query client
  api.ts      // All API endpoints using the wrapped client
```

client.ts (API Client Layer)

- `client.ts` is the **single, centralized API client**
- It is responsible for:
    - Axios instance creation
    - Base URL configuration
    - Request and response interceptors
    - Authentication headers
    - Global error handling

Rules:

- Axios (or Fetch) must be used **only inside `client.ts`**
- No other file is allowed to create or configure HTTP clients
- No business logic is allowed in this file

---

api.ts (Endpoint Definition Layer)

- `api.ts` must contain **all API endpoint definitions**

- All endpoints must use the wrapped client from `client.ts`
- Endpoints must be:
    - Clearly named
    - Grouped logically (auth, user, product, etc.)
    - Typed (request and response)

Rules:

- No direct Axios or Fetch usage inside `api.ts`
- No API logic inside UI or container components
- No additional service files are allowed

---

10. Error Handling

Error handling must be **centralized, predictable, and user-safe**.
The application must never crash silently or expose internal error details to users.

10.1 Error Boundaries

Error Boundaries are used to catch **runtime UI errors** and prevent the entire app from crashing.

Rules

- A **global Error Boundary is mandatory** at the application root
- Feature-level Error Boundaries are allowed for critical modules
- Error Boundaries **must not** contain business logic
- Error Boundaries **must not** retry API calls automatically

Responsibilities

- Catch rendering errors
- Show fallback UI
- Log the error for monitoring

Example Responsibilities

- Display a generic “Something went wrong” screen
- Provide a refresh or navigation option
- Capture error stack trace for logging

10.2 API Errors

API errors must be handled **outside UI components** and normalized before reaching the UI.

Rules

- All API calls must go through a **central API client**
- API error handling must be implemented using:
    - Interceptors (Axios) or
    - Central fetch wrapper
- UI components must **never parse raw API error responses**

Standard API Error Categories

- **400** – Validation / bad request
- **401** – Unauthorized → logout or token refresh
- **403** – Forbidden → access denied screen
- **404** – Resource not found
- **500+** – Server error

Mandatory Practices

- Normalize API error shape before returning
- Map technical errors to user-friendly messages
- Handle auth-related errors globally

10.3 Toast Handling

Toasts are used only for **user-visible, short-lived feedback**.

Allowed Use Cases

- API success messages
- API failure messages
- Form submission feedback
- Non-blocking warnings

Rules

- Toast logic must be centralized (utility or hook)
- UI components must not hardcode toast messages
- Avoid duplicate toasts for the same event
- No toast spamming (rate-limit repeated failures)

Guidelines

- Success → short and clear
- Error → user-friendly, no technical jargon
- Critical errors → modal or error screen (not toast)

10.4 Logging

Logging is for **developers and monitoring systems**, not users.

Rules

- `console.log`, `console.error` are **forbidden in production**
- Use a centralized logging utility
- Log only actionable and meaningful data
- Never log:
    - Tokens
    - Passwords
    - Personal or sensitive user data

What to Log

- Application crashes
- Unhandled exceptions
- API failures (with metadata)
- Performance issues (optional)

Logging Levels

- `info` – lifecycle events
- `warn` – recoverable issues
- `error` – crashes and failures

---

11. Custom Hooks Guidelines

Custom hooks are used to **extract reusable logic** and **share behavior across components**. They must remain **pure, predictable, and UI-agnostic**.

11.1 Naming

Rules

- Hook names **must start with** `use`
- Use **camelCase** after `use`
- Name must clearly describe behavior or intent

Examples

- `useAuth`
- `useDebounce`
- `useUserPermissions`

`File Naming`

- `File name must match kebab-case (use-auth.ts, use-debounce.ts, etc.)`
- `One hook per file`

11.2 Reusability

Hooks must be **generic and reusable** by default.

Rules

- Hooks must not depend on a specific page or feature
- Avoid hardcoded values
- Accept configuration via parameters
- Hooks should return:
    - State
    - Derived data
    - Action handlers

Good Practice

- Prefer small, composable hooks
- Split large hooks into multiple focused hooks
- Hooks should work in **any component context**

11.3 No UI Logic

Custom hooks must contain **zero UI logic**.

Strict Rules

- No JSX
- No HTML elements
- No styling logic
- No direct DOM manipulation

Allowed Inside Hooks

- State management
- Side effects (`useEffect`)
- API calls (via services)
- Data transformation
- Event handlers

Not Allowed Inside Hooks

- Rendering components
- Opening modals
- Showing toasts directly

- Navigating routes directly

Hooks provide data and actions, and UI decides how to present them.

11.4 Testing Hooks

Custom hooks must be **testable and predictable**.

Rules

- Hooks with business logic **must have tests**
- Hooks must not rely on hidden globals
- External dependencies must be mockable

Testing Guidelines

- Test behavior, not implementation
- Cover:
    - Initial state
    - State updates
    - Side effects
    - Error handling

Recommended Tools

- Jest or Vitest (Recommended)
- React Testing Library (`renderHook`)

---

12. Business Logic & UI Separation

This architecture separates **what the app does**, **how screens are coordinated**, and **how UI is rendered**.
The goal is to keep logic testable, UI reusable, and screens easy to reason about.

12.1 Business Logic

Business Logic defines **data handling, rules, and state management**.
It includes fetching data, transforming responses, validations, and managing shared state.
In this project, business logic lives **inside container hooks and store files**, but **never in JSX**.

```
containers/user-management/
├─ users/
│  └─ use-users.ts
├─ add-edit-user/
│  └─ use-add-edit-user.ts
└─ use-user-store.ts
```

12.2 Container Layer (Screen coordination)

The Container Layer coordinates screens.
It connects business logic to UI components, handles lifecycle events, and passes prepared data to UI.
Containers compose screens but do not contain business rules or UI-specific rendering logic.

```
containers/user-management/
├─ users/
│  └─ index.tsx
├─ add-edit-user/
│  └─ index.tsx
```

12.3 UI Layer (Pure UI)

The UI Layer contains **pure presentational components**.
Components only render data received via props and emit user interactions via callbacks.
They do not access APIs, stores, or business logic directly.

```
components/user-management/
├─ user-actions.tsx
└─ user-card.rtsx
```

Golden Rule:

- All data fetching must live inside container hooks (`use-users.ts`, `use-add-edit-user.ts`)
- UI components must not access stores directly
- UI components must not import container hooks
- Pages must only render containers, nothing else
- State management must be handled via hooks or stores
- UI components receive data and callbacks via props only
- Each screen must have a single container entry point
- Hooks must return data and handlers, never JSX

---

13. UI & Theming

13.1 Styling System

- The project must use a **single, standardized styling system**.
- Allowed styling approaches:
    - Tailwind CSS (Latest version recommended)
    - CSS Modules
    - Styled Components (only if approved)

- Mixing multiple styling systems in the same project is **not allowed**.
- Inline styles should be avoided except for dynamic values.

13.2 Theme Provider

- A **central Theme Provider** is mandatory.
- The theme provider must:
    - Control colors, typography, spacing, and shadows
    - Be initialized at the application root
- UI components must consume theme values instead of hardcoded styles.

13.3 Design Tokens

- Design tokens must be defined centrally.
- Tokens include:
    - Colors
    - Font sizes
    - Spacing
    - Border radius
    - Z-index levels
- Tokens must be reusable and consistent across the application.
- Hardcoded UI values inside components are **not allowed**.

13.4 Dark Mode

- Dark mode support is recommended for all new projects.
- Theme switching must be handled via the Theme Provider.

- Components must not contain hardcoded light or dark colors.
- User preferences must be respected and persisted.

---

14. Common Components

Common components are **pure UI building blocks** used across the application.

- Buttons
- Inputs
- Tables
- Modals

Example:

```
components/ui
├─ input.tsx
├─ table.tsx
├─ avatar.tsx
└─ button.tsx
```

Rules & Responsibility

- Components must be **fully reusable**
- No business logic inside common components
- No API calls inside components
- Common components handle **presentation only**
- Data fetching and state management belong to containers
- Validation and side effects must be handled outside UI components

- Props must be clearly typed
- Styling must rely on the theme and design tokens
- Components must remain framework-agnostic where possible

---

15. Performance Optimization

Performance optimization must be considered **by default** for all React applications, especially for large-scale and user-facing systems.

15.1 Lazy Loading

- Route-level components must be lazy-loaded using `React.lazy` and `Suspense`.
- Heavy components, modals, and dashboards should be loaded only when required.
- Avoid loading non-critical components during initial page load.

Rules:

- Lazy loading is mandatory for routes and large feature modules.
- Suspense fallback components must be lightweight.

15.2 Code Splitting

- Code splitting must be implemented at:
    - Route level
    - Feature/module level when applicable
- Avoid bundling unrelated features into a single chunk.
- Dynamic imports must be used responsibly to reduce bundle size.

15.3 Memoization

- Use `useMemo` and `useCallback` to prevent unnecessary recalculations and re-renders.
- Memoization must be applied **only when there is a proven performance benefit**.
- Overusing memoization without measurement is discouraged.

Rules:

- Memoized values must be deterministic.
- Avoid using impure functions during render cycles.

15.4 Virtualization

- Virtualization is mandatory for large lists and tables.
- Use libraries such as:
    - `@tanstack/react-virtual (Recommended for Tailwindcss + Shadcn)`
    - `react-window`
    - `React-virtuoso`
- Rendering all items at once for large datasets is **not allowed**.

15.5 Monitoring & Measurement

- Performance must be measured using:
    - React DevTools Profiler
    - Browser Performance Tools
- Optimizations should be data-driven, not assumption-based.

---

16. Security Guidelines

Security rules apply across **containers**, **components**, and **pages**.
Violations must be treated as **blockers in code review**.

16.1 Token Storage

- Tokens must **not** be stored in UI components
- Avoid storing sensitive tokens in `localStorage` unless explicitly required
- Prefer **HttpOnly cookies** for authentication tokens
- If client-side storage is unavoidable:
    - Store only non-sensitive identifiers
    - Never store refresh tokens
- Never log tokens or auth headers

16.2 XSS Prevention

- Avoid `dangerouslySetInnerHTML`
- Never render raw HTML from APIs without sanitization
- Sanitize any dynamic HTML content before rendering
- Rely on React's default escaping for JSX
- Do not trust user input or API responses blindly

16.3 API Protection

- No API calls inside UI components (`/components`)
- No API calls inside pages (`/pages`)
- API calls must be made only from:

    - Container hooks
    - Centralized API utilities
- Handle authorization and error responses centrally
- Do not expose internal API URLs or secrets in UI code

16.4 Environment Variables

- Never commit `.env` files with real values
- Maintain `.env.example` for reference
- Do not expose secrets to client-side code
- Use environment variables only for:
    - Public configuration
    - Environment-specific flags
- Do not hardcode environment values in code

---

17. Testing Standards

Testing is mandatory for **business logic and containers**. UI components should be tested where behavior matters.

17.1 Jest or Vitest (Recommended)

- Test business logic, hooks, stores, and containers directly
- No snapshot-only tests
- Focus on user behavior and interactions
- Mock external dependencies and APIs

- Do not test implementation details

17.2 React Testing Library

- Use React Testing Library for **component and container tests**
- Test **user behavior**, not component internals
- Prefer queries like:
    - `getByRole`
    - `getByText`
    - `getByLabelText`
- Avoid testing internal state or private methods